Prakash Thapa
D15A -27

Experiment 04
Implementation of K-Nearest Neighbors (KNN) and Model Performance Evaluation

Dataset: Iris Dataset

---

1. Dataset Source

Dataset Name: **Iris Dataset**

Source: Kaggle
Link: https://www.kaggle.com/datasets/uciml/iris

The Iris dataset is a widely used dataset in machine learning for classification tasks. It contains measurements of iris flowers belonging to three different species and is commonly used for evaluating classification algorithms.

2. Dataset Description

The dataset consists of **150 samples** of iris flowers with **4 numerical features** and **1 target variable**.

Input Features

Feature	Description
sepal_length	Length of the sepal (cm)
sepal_width	Width of the sepal (cm)
petal_length	Length of the petal (cm)
petal_width	Width of the petal (cm)

Target Variable

Variable	Description
Species	Type of iris flower (Setosa, Versicolor, Virginica)

Dataset Characteristics

- Total records: **150**
- Number of features: **4**
- Number of classes: **3**
- Balanced dataset with equal samples per class
- No missing values

The objective of the experiment is to classify iris flowers into their respective species using the K-Nearest Neighbors algorithm.

3. Mathematical Formulation of the Algorithm

The K-Nearest Neighbors algorithm is a **non-parametric supervised learning algorithm** used for classification and regression.

The algorithm classifies a new data point based on the majority class among its **K nearest neighbors**.

Distance between data points is typically measured using **Euclidean Distance**:

$$d(x, y) = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$

Where:

- xx = test data point
- yy = training data point
- nn = number of features

The algorithm steps are:

1. Choose the number of neighbors **K**

2. Compute the distance between the test point and all training points
3. Select the **K nearest neighbors**
4. Assign the class that occurs most frequently among those neighbors

4. Algorithm Limitations

Although KNN is simple and effective, it has several limitations:

1. **Computational Cost**
   KNN requires calculating distances to all training samples during prediction, making it slow for large datasets.
2. **Sensitive to Feature Scaling**
   Features with larger magnitudes can dominate the distance calculation.
3. **Choice of K Value**
   A very small K may lead to overfitting, while a very large K may reduce model accuracy.
4. **Memory Requirement**
   Since KNN stores the entire training dataset, it requires more memory compared to some other models.

5. Methodology / Workflow

The following steps were performed in the experiment.

Step 1: Data Loading

The Iris dataset was loaded using the Pandas library.

[1]
✓ 1s

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

Step 2: Data Exploration

Basic dataset analysis was performed to examine:

- dataset shape
- column names
- data types
- missing values
- class distribution

[3]
✓ 0s

```
df = pd.read_csv("Iris.csv")

df.head()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

[4]
✓ 0s

```
# Check dataset shape
print("Dataset Shape:", df.shape)

# Column names
print("Columns:", df.columns)

# Basic information
df.info()

# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Check class distribution
print("\nClass Distribution:")
print(df["Species"].value_counts())
```

```
Dataset Shape: (150, 6)
Columns: Index(['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm',
       'Species'],
      dtype='object')
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             150 non-null    int64
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   Species        150 non-null    object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB

Missing Values:
Id               0
SepalLengthCm    0
SepalWidthCm     0
PetalLengthCm    0
PetalWidthCm     0
Species          0
dtype: int64

Class Distribution:
Species
Iris-setosa        50
Iris-versicolor    50
Iris-virginica     50
Name: count, dtype: int64
```

Step 3: Data Preprocessing

The unnecessary **Id column** was removed.
Features and target variables were separated.

[5]
✓ 0s

```python
# Drop the Id column
df = df.drop("Id", axis=1)

# Separate features and target
X = df.drop("Species", axis=1)
y = df["Species"]

print("Feature Shape:", X.shape)
print("Target Shape:", y.shape)
```

```
Feature Shape: (150, 4)
Target Shape: (150,)
```

Step 4: Train-Test Split

The dataset was divided into:

- 80% training data
- 20% testing data

```
[6]
✓ 0s
    X_train, X_test, y_train, y_test = train_test_split(
        X, y,
        test_size=0.2,
        random_state=42
    )

    print("Training Data Shape:", X_train.shape)
    print("Testing Data Shape:", X_test.shape)

    Training Data Shape: (120, 4)
    Testing Data Shape: (30, 4)
```

Step 5: Feature Scaling

StandardScaler was applied to normalize the features because KNN relies on distance calculations.

```
[7]
✓ 0s
    scaler = StandardScaler()

    # Fit on training data and transform
    X_train_scaled = scaler.fit_transform(X_train)

    # Transform test data
    X_test_scaled = scaler.transform(X_test)

    print("Scaled Training Data Shape:", X_train_scaled.shape)
    print("Scaled Testing Data Shape:", X_test_scaled.shape)

    Scaled Training Data Shape: (120, 4)
    Scaled Testing Data Shape: (30, 4)
```

Step 6: Model Training

A KNN classifier was implemented with **K = 5 neighbors**.

[8] ✓ 0s

```
knn = KNeighborsClassifier(n_neighbors=5)

# Train the model
knn.fit(X_train_scaled, y_train)

print("KNN model training completed")
```

```
KNN model training completed
```

Step 7: Prediction

The trained model was used to predict the class labels for the testing dataset.

[9] ✓ 0s

```
y_pred = knn.predict(X_test_scaled)

print("Predicted Values:")
print(y_pred)
```

```
Predicted Values:
['Iris-versicolor' 'Iris-setosa' 'Iris-virginica' 'Iris-versicolor'
 'Iris-versicolor' 'Iris-setosa' 'Iris-versicolor' 'Iris-virginica'
 'Iris-versicolor' 'Iris-versicolor' 'Iris-virginica' 'Iris-setosa'
 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-versicolor'
 'Iris-virginica' 'Iris-versicolor' 'Iris-versicolor' 'Iris-virginica'
 'Iris-setosa' 'Iris-virginica' 'Iris-setosa' 'Iris-virginica'
 'Iris-virginica' 'Iris-virginica' 'Iris-virginica' 'Iris-virginica'
 'Iris-setosa' 'Iris-setosa']
```

Step 8: Model Evaluation

Model performance was evaluated using:

- Accuracy score
- Confusion matrix
- Classification report

[10] ✓ 0s

```
# Accuracy of the model
accuracy = accuracy_score(y_test, y_pred)
print("Model Accuracy:", accuracy)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:\n", cm)

# Classification Report
report = classification_report(y_test, y_pred)
print("\nClassification Report:\n", report)
```

```
Model Accuracy: 1.0

Confusion Matrix:
 [[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]

Classification Report:
                 precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        10
Iris-versicolor       1.00      1.00      1.00         9
 Iris-virginica       1.00      1.00      1.00        11

       accuracy                           1.00        30
      macro avg       1.00      1.00      1.00        30
   weighted avg       1.00      1.00      1.00        30
```

Step 9: Visualization

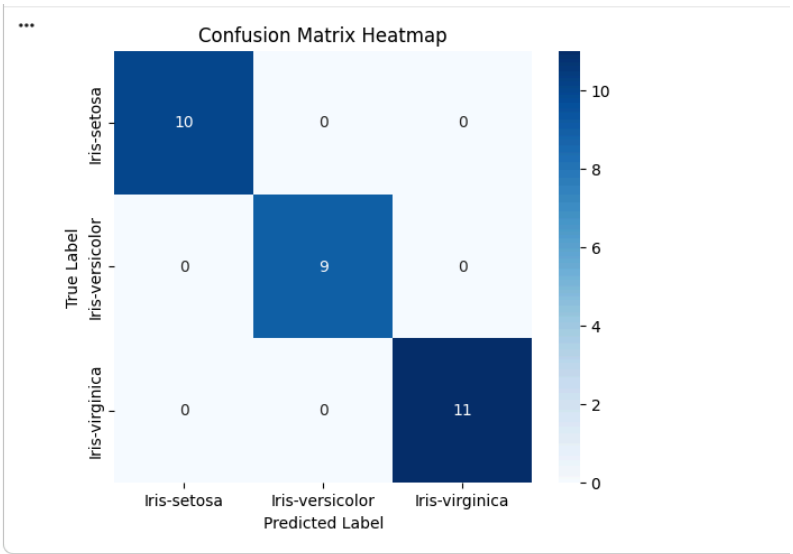
Visualizations were created including:

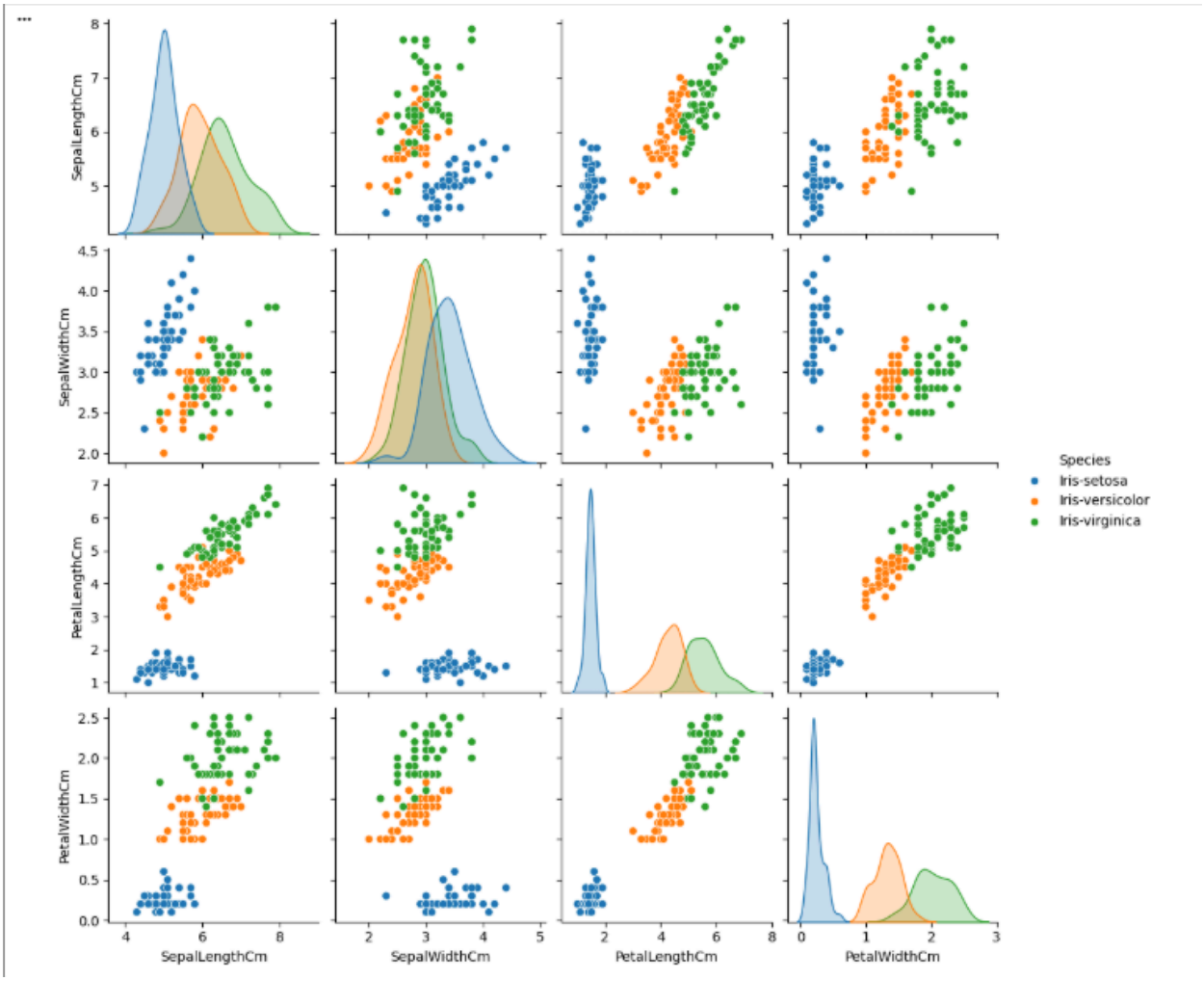
- Confusion matrix heatmap
- Pairplot to visualize feature relationships between species

```
[11]
✓ 0s
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, cmap="Blues",
            xticklabels=knn.classes_,
            yticklabels=knn.classes_)

plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix Heatmap")
plt.show()
```





6. Performance Analysis

The performance of the KNN model was evaluated using classification metrics.

Accuracy

Accuracy measures the proportion of correct predictions.

$$Accuracy = \frac{Correct\ Predictions}{Total\ Predictions}$$

The KNN classifier achieved **high accuracy** on the Iris dataset due to clear separation between flower species.

Confusion Matrix

The confusion matrix shows correct and incorrect predictions for each species class.

Most predictions were located along the diagonal of the matrix, indicating correct classification.

Classification Report

The classification report includes:

- Precision
- Recall
- F1-Score

These metrics showed strong performance for all three iris species.

The **Setosa class was predicted with the highest accuracy** due to its distinct feature patterns.

7. Hyperparameter Tuning

The most important hyperparameter in KNN is the **number of neighbors (K)**.

Different K values can significantly affect model performance.

Example values tested:

K = 3, 5, 7, 9

Hyperparameter tuning can be performed using techniques such as:

- GridSearchCV
- Cross-Validation

Optimal K value helps balance bias and variance in the model.

8. Conclusion

The experiment successfully implemented the K-Nearest Neighbors algorithm for classifying iris flower species.

The model achieved strong performance due to the well-structured and balanced nature of the dataset. Feature scaling played an important role in improving model accuracy because KNN relies on distance-based calculations.

Visualization techniques such as pairplots and confusion matrix heatmaps helped in understanding feature relationships and evaluating classification performance.

Overall, the experiment demonstrates the effectiveness of KNN for classification problems with small to medium-sized datasets and well-separated feature spaces.